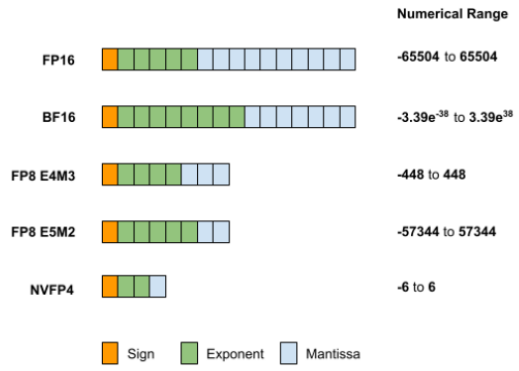
Cuantización

(q = cuantizacion)
Existen 3 formas de cuantizar los modelos

- post traning dynamic quantization
- post traning static quantization
- quatizacion aware training

Un numero en punto flotante utiliza n bits para almacenar un numero,
son `sign` es el signo de un bit indica el numero 0 por positivo y 1 por negativo. `Exponent` almacena el exponenete, representa la potencia referida a base 2 comunmente.
`Significand/mantissa` números de dígitos significativos

formula de representación x `=(-1)^sign × 2^exponent ×mantissa`



Elementos que se cuantizan para aumentar la rapidez y eso.

- Cuantizar los pesos del modelo para reducir la memory footprint
- Activaciones de modelo (Model Activations), son las salidas intermedias del modelo, generadas después de cada operación durante la inferencia. Las activaciones suelen de ser dinámicas y no incluidas explícitamente en el modelo.
- KV caches (applicable only to decoder models).

Algoritmos de cuantizacion

- **Affine quantization compared to symmetric quantization**

Affine quantization: se define por dos parámetros 1. el factor de escala *s*
y el punto cero *z*.

- *s* factor de escala: numero en punto flotante que determine el tamaño del step de cuantizacion.
- *z* cero point: es del mismo tipo que el valor cuantizado y asegura que el valor real `cero` se mapee exactamente en el proceso de cuantizacion. Este parámetro asegura la eficiencia de implementación cuando se realiza el padding con ceros (el rellano con

ceros en los bordes de los arreglos)

$$x_q = \text{clip}\left(\text{round}\left(\frac{x}{s} + z\right), \alpha_q, \beta_q\right)$$

Where:

- x is the original value
- x_q is the quantized value
- s is the scale factor, a positive real number
- z is the zero point
- α_q, β_q define the range for the quantized representations
- round converts the scaled value to the nearest quantized representation
- clip ensures values stay within the range of quantized representations

- To recover the approximate full-precision value from a quantized value:

$$x \approx x' = s \cdot (x_q - z)$$

Symmetric quantization
Es un caso simplificado del caso asymmetric donde el punto cero se modifica por 0. Reduce el costo computacional porque se reducen operaciones adicionales.

AbsMax algorithm

Determina como se determina el calor de *s* el factor de escala. Como se calcula la cuantización. $s = \frac{\max(|x_{\max}|, |x_{\min}|) \cdot 2}{\beta_q - \alpha_q}$

Es el mas básico, surgieron muchos algoritmos avanzados.

- Activation-aware Weight Quantization (AWQ)g: AWQ Es solo de pesos que protege una parte pequeña de los pesos de los canales salientes (a parentemente son los mas criticos).
- Generative Pre-trained Transformer Quantization (GPTQ): Comprime el modelo cuantizandolo para cada fila de pesos de la matriz independientemente. Utiliza información de segundo orden. La matriz hessiana para guiar el proceso de cuantización.
- SmoothQuant: SmoothQuant enables both weights and activations to be quantized to 8 bits by applying a mathematically equivalent per-channel scaling transformation that smooths out activation outliers, shifting quantization difficulty from activations to weights and preserving model accuracy and hardware efficiency.

Quantization granularity

Explica como los parámetros de cuantización son compartidos a través de los elementos de un tensor. Refiere al nivel de granularidad con el que calculamos los valores de x_max y x_min de los datos originales en la cuantización AbsMax.

Granularity --> implica cuantos elementos de un mismo tensor comparten los mismos parámetros de cuantización. Es decir sobre que grupo de valores se computa el máximo o mínimo para obtener la escala.

- Por tensor (o por capa): todos los valores del tensor utilizan el mismo set de cuantización. Este es **el mas eficiente en uso de memoria y simple**. Suele tener errores en valores altos de q. Especialmente cuando la distribución de los datos varia acrross de las dimensiones (diferentes slides de un tensor tiene una gran variación del rango)
- Por canal: Diferente parámetros son usados para cada canal.
- Por bloque (o por grupo):

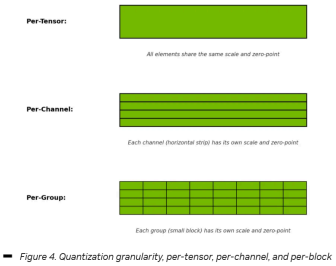


Figure 4. Quantization granularity, per-tensor, per-channel, and per-block

Quantization approaches

- **post training quantization**: puede ser solo de pesos o de pesos y activaciones.
  - Solo pesos --> solo se necesita info de los pesos
  - Ambos --> Se necesecita input data, debido a que la activaciones solo se obtienen teniendo input data.
  - El proceso de recolectar activaciones estáticas y determinar valores apropiedados de scale y cero point es conocido como calibracion.
  - Static q: involucra el uso de un dataset de calibración para computar los parametros. Estos parametros luego son usados para todas las futuras inferencias
  - Dinámico, los parámetros se computan durante la inferencia, los parámetros pueden ser calculados durante el vuelo y varían para cada entrada. No requiere un dataset de calibración. La cuantización de los pesos no se modifica (estatica), la que si se modifica es la de las activaciones (runtime)
- Aware training (QAT): aplica la cuantización durante el entrenamiento Esto se logra simulando una aritmética de baja precisión tanto en la pasada hacia adelante (forward pass) como en la pasada hacia atrás (backward pass), lo que permite que los parámetros del modelo aprendan y se adapten a los errores derivados de la cuantización, tales como los errores de redondeo y de truncamiento (clipping).
  - **fake quantization**: introduce módulos de mímica, que simulan el comportamiento de baja precisión sin alterar los tipos de datos actuales. Es decir cuantiza mide y decuantiza.

- Se usa STE straight through estimator, aproxima el gradiente de esta funciones como una identidad durante el backpropagation, facilitando el entrenamiento a presar de que halla operaciones no diferenciables. Por ejemplo con la función round() o con clip
- ∂x_fq/∂x ≈ 1 if x is inside the range [min, max] ∂x_fq/∂x = 0 if x was clipped

Quark AMD:

Es solamente un cuantizador, una librería de software que toma un modelo en float y te devuelve un modelo cuantizado (pesos INT8 + escalas). (es de amd pero esta en las generaciones mas nuevas de la versal.)
Si implemento yo misma la inferencia en la Versal puedo usar QUARK para generar el modelo cuantizado.
Mini esquema de implementación manual:
1. Cuantizar con Quark en Pc (pytorch)
2. Exportar el modelo cuantizado, sale como ONNX(Open Neural Network Exchange) con nodos.
3. Extraer los pesos INT8, los baias las escalas de ese ONNx y generar headers C/C++.
4. Escribir los kernels en HLS o con la AIE APi para los Ai engines.
5. Verificar bit a bir contra la simulacion Quark (no se bien que es esro.)
Sin emabargo tiene mas peros que otra cosa --> no es util !!

vai_q_pytorch

vai_q_pytorch is short for Vitis AI Quantizer for Pytorch (librería de vitis)
Workflow:

1. **Genera el grafo del modelo** con un input de ejemplos (con `dummy_input` se lo paso al `torch_quantizer`). Genera un grafo de computo en una representación intermedia (IR). El IR es un formato neutro, es como el lenguaje intermedio entre el origen y el destino. Aca es donde suelen fallar los modelos con control de flujo dinámico -> por eso existen los custom ops.
2. Modificar el grafo computacional que se hizo antes, se hace el pruning, cuantizacion y optimizacion
    1. **Optimización del grafo:**
    - *BN folding*: fusionar BatchNorm dentro de la conv anterior, ajustando pesos y bias.
    - *Fusión de capas*: conv + ReLU como un solo nodo, porque el DPU las ejecuta juntas.
    - Eliminar nodos inútiles y reemplazar ops no soportadas por equivalentes (por ejemplo, ReLU6 por ReLU, si se configura).
    2. **Cuantización:** insertar los nodos de cuantización (fake quant) en pesos y activaciones, correr la calibración para fijar las escalas (las *fix positions*, potencia de 2) y, si corresponde, fast finetuning o QAT.

    3. **Pruning:** eliminar canales o pesos poco importantes. Lo hace el Optimizer, pero opera sobre la misma IR.
3. Desplegar el IR modificado en diferentes plataformas como DPU y Xilinx AI Engin.
    1. `export_xmodel()` lo traduce a **XIR** (el `.xmodel`), la IR que entiende el **compilador de Vitis AI**.
    2. Después, `vai_c_xir` compila ese xmodel para tu DPU concreto (el DPUCVDX8G en la VCK190): decide qué subgrafos van al DPU y cuáles a la CPU, y genera las instrucciones.
4. Volver a ensamblar el grafo modificado en PyTorch. La herramienta puede renegerar el modelo de Pytorch a partir de IR modificada. En `vai_q_pytorch` esto es literal: en la carpeta `quantize_result/` aparece un archivo `.py` con un `nn.Module` generado automáticamente, que es tu red ya fusionada y con los nodos de cuantización.
   NO SOPORTA DATA PARALLELISM

https://codesignp211.informatik.uni-erlangen.de:450/kilic/qnn_project_w2021/-/blob/master/Vitis-AI/tools/Vitis-AI-Quantizer/vai_q_pytorch/README.md?ref_type=heads#add-vai_q_pytorch-apis-to-float-scripts

quantize.py

Se encuentra la configuración para llevar a cabo la cuantización. Desde la terminal se pueden insetar los comandos para que cuantise de distintas maneras sin modificar el código `argparse`.

```
  ap.add_argument('-q',  '--quant_mode', type=str, default='calib',
choices=['calib','test'], help='Quantization mode (calib or test). Default
is calib')
```

Dentro del script se crea el modelo de grafos,

```
rand_in = torch.randn([batchsize, 1, 28, 28])
quantizer = torch_quantizer(quant_mode, model, (rand_in),
output_dir=quant_model)
```

y con el comando `quantizer.export_model` crea el grafo para el hardware, `export_quant_config` no crea el grafo sino que guarda los números de calibración.

Cuantización parcial

Desde el archivo de quantize.py se puede setear el estilo/forma de cuantizar.

Objetivo: cuantizar el modulo subm0 y subm1 sin cuantizar el modulo subm1.

```
from pytorch_nndct.nn import QuantStub, DeQuantStub

class WholeModule(torch.nn.Module):
    def __init__(self, ...):
        self.subm0 = ...
        self.subm1 = ...
        self.subm2 = ...

        # Define QuantStub/DeQuantStub submodules
        self.quant = QuantStub()
        self.dequant = DeQuantStub()

    def forward(self, input):
        input = self.quant(input)  # Begin of part to be quantized
        output0 = self.subm0(input)
        output0 = self.dequant(output0)  # End of part to be quantized

        output1 = self.subm1(output0)

        output1 = self.quant(output1)  # Begin of part to be quantized
        output2 = self.subm2(output1)
        output2 = self.dequant(output2)  # End of part to be quantized
```

`

Cuando la accuracy no es suficiente se realiza un finetune de los parámetros del modelo. (Es lo que mencioné antes de "*fake quantization*" , el cual no es un entrenamiento real del modelo. Necesita solo un número limitado de iteraciones depende del modelo.)
La re calibración con el modelo original es recomendada

```
# fast finetune model or load finetuned parameter before test
if finetune == True: ft_loader, _ = load_data(
     subset_len=1024,
     train=False,
     batch_size=batch_size,
     sample_method=None,
     data_dir=args.data_dir,
     model_name=model_name)
     if quant_mode == 'calib'
          quantizer.fast_finetune(evaluate, (quant_model,
          ft_loader, loss_fn))
     elif quant_mode == 'test':
          quantizer.load_ft_param()`
```

FineTune - QAT

`qat_proc = True` Le dice al quantizer que prepare el modelo para **entrenar**, no solo para calibrar.'
Creo que al script le faltaría un modulo de entrenamiento.

```
`# vai_q_pytorch interface function: create quantizer can do QAT

 input = torch.randn([batch_size, 3, 224, 224], dtype=torch.float32)

 quantizer = torch_quantizer(quant_mode = 'calib',
     module = model, input_args = input, bitwidth = 8,
     mix_bit = False, qat_proc = True)



quantized_model = quantizer.quant_model`


`# vai_q_pytorch interface function: deploy the trained model and convert
xmodel need at least 1 iteration of inference with batch_size=1

quantizer.deploy(quantized_model)

deployable_model = quantizer.deploy_model
val_dataset2 = torch.utils.data.Subset(val_dataset, list(range(1)))
     val_loader2 = torch.utils.data.DataLoader( val_dataset2, batch_size=1,
     shuffle=False, num_workers=workers, pin_memory=True)
validate(val_loader2,
     deployable_model, criterion, gpu)
quantizer.export_xmodel()`
```